Session 9

Conditional Logic & Nullability in Kotlin

Student Name: Guru K R

Course: Android App Development using Gen AI

Date: March 2026

Section 1: Advanced Conditional Logic & Flow Design

1.1 Definitions

if/else Expression vs Statement

In most languages, if/else is a statement — it executes code but returns nothing. In Kotlin, if/else is also an expression, meaning it produces a value that can be assigned to a variable. When used as an expression, every branch must return a value of the same type. When used as a statement (without assignment), the return value is simply discarded. This dual nature eliminates the need for a ternary operator in Kotlin.

when Statement vs when Expression

The when construct is Kotlin's replacement for switch. As a statement, when executes the matching branch and discards any return value — useful for triggering side effects. As an expression, when must cover all possible inputs (either with explicit branches or an else branch) and returns the value of the matched branch. Using when as an expression forces exhaustive handling of all cases, making the code more robust.

Use of 'in' Range in when

The in keyword inside a when block checks whether a value falls within a range, such as 0..100 or a collection. This is equivalent to writing value >= 0 && value <= 100 but is far more readable. Kotlin ranges are inclusive by default, and until can be used for exclusive upper bounds (e.g. 0 until 100 means 0 to 99).

Use of 'is' Type Check in when

The is keyword performs a runtime type check and, crucially, triggers a smart cast — Kotlin automatically treats the variable as the checked type within that branch, eliminating the need for an explicit cast. This makes when expressions powerful for handling sealed class hierarchies or Any types safely and concisely.

1.2 Kotlin Code Examples

if/else Expression Returning a Value

```
// if/else as an expression — assigns the result directly to discount
val amount = 150
val discount = if (amount > 100) {
    10  // returned when condition is true
} else {
    0   // returned when condition is false
}
println("Discount: $discount%")  // Output: Discount: 10%
```

when Expression with Range and Type Check

```
fun categorizeValue(value: Any): String {
    return when (value) {
        in 0..100        -> "Low range"
```

```
        in 101..500      -> "Medium range"
        is String        -> "String value: $value"  // smart cast applies
        is Double        -> "Double value: $value"
        else             -> "Unknown type or out of range"
    }
}

println(categorizeValue(75))       // Low range
println(categorizeValue(250))      // Medium range
println(categorizeValue("hello"))  // String value: hello
```

1.3 Feature Example — Expense Categorizer

Feature Name: Expense Categorizer

This feature categorizes user expenses and applies discounts or surcharges based on the transaction amount and the user's account type. It demonstrates practical use of if/else expressions, when ranges, and Boolean conditions in a real-world financial context.

Decision Rules:

- If amount < 0 → reject the transaction immediately (invalid input)
- If amount is in 0..100 → assign Category A (small purchase, no modifier)
- If amount > 100 and user is premium → apply a 10% discount
- Otherwise → apply a 5% surcharge (large purchase, non-premium user)

```
fun categorizeExpense(amount: Double, isPremium: Boolean): String {
    if (amount < 0) return "Error: invalid transaction amount"

    return when {
        amount in 0.0..100.0          -> "Category A — small purchase, no modifier"
        amount > 100.0 && isPremium   -> {
            val discounted = amount * 0.90
            "Premium discount applied. Final: $${'$'}discounted"
        }
        else -> {
            val surcharge = amount * 1.05
            "Surcharge applied. Final: $${'$'}surcharge"
        }
    }
}

// Usage
println(categorizeExpense(50.0, false))   // Category A
println(categorizeExpense(200.0, true))   // Premium discount applied
println(categorizeExpense(200.0, false))  // Surcharge applied
```

1.4 Decision Flow Table

Condition	Action	Kotlin Logic	Result / Output

amount < 0	Reject transaction	if (amount < 0) return ...	"Error: invalid transaction amount"
amount in 0..100	Assign Category A	amount in 0.0..100.0 (when branch)	"Category A — small purchase"
amount > 100 && isPremium	Apply 10% discount	amount > 100.0 && isPremium (when branch)	Final = amount * 0.90
else (large, non-premium)	Apply 5% surcharge	else branch in when expression	Final = amount * 1.05

[Insert flow diagram of Expense Categorizer decision logic here]

1.5 Jetpack Compose Integration Plan

The Expense Categorizer logic integrates with Jetpack Compose through a reactive state pipeline. A TextField composable captures the user's expense amount as a String state variable (amountInput). An operator or category is derived on every recomposition using categorizeExpense(amountInput.toDoubleOrNull() ?: -1.0, isPremium). The result string is displayed in a Text composable below the input, updating automatically as the user types. A premium toggle Switch bound to a Boolean state variable allows the user to switch their account type, causing the category and final value to recalculate immediately without any manual refresh.

Flow: User types amount → amountInput state updates → categorizeExpense() re-evaluates → category Text recomposes → user sees updated result.

1.6 Reflection — Why Planning Decision Logic Improves Maintainability

Planning conditional logic before coding forces a complete enumeration of every possible state and input combination before a single line of implementation is written. In the Expense Categorizer, thinking through the decision rules as a table first revealed that the boundary between Category A and the premium/surcharge branches is inclusive at 100.0 — a detail that is easy to get wrong if conditions are written ad hoc. When decision logic is planned upfront, it becomes a shared specification that other developers or testers can read and verify independently of the code. Adding a new rule — for example, a new 'Category B' for amounts between 101 and 200 — requires only inserting a new row in the table and a new when branch, without needing to understand the entire implementation. Unplanned conditional logic tends to accumulate as nested if/else chains that are difficult to read, test, and extend, while structured when expressions derived from a clear decision table are self-documenting and trivially unit-testable.

Section 2: Nullability & Safe Data Handling

2.1 Kotlin Null-Safety Definitions

Nullable Types (String?)

A nullable type is declared by appending ? to the type name (e.g. String?). This means the variable is allowed to hold either a value of that type or null. Kotlin's type system enforces at compile time that nullable types must be explicitly handled before they can be used, preventing unintended null pointer exceptions.

Non-nullable Types (String)

A non-nullable type (e.g. String without ?) can never hold null. The Kotlin compiler guarantees this at compile time — assigning null to a non-nullable variable is a compile error. This removes an entire category of null-related bugs that are common in Java.

Safe Call Operator (?.)

The safe call operator (?.) calls a method or accesses a property only if the receiver is not null. If the receiver is null, the entire expression short-circuits and returns null instead of throwing a NullPointerException. It can be chained for multiple levels of potentially null objects.

Elvis Operator (?:)

The Elvis operator (?:) provides a default value when the left-hand expression is null. It is equivalent to: if (expr != null) expr else defaultValue. It is named after Elvis Presley because the ?: symbol resembles his famous hairdo when rotated.

Non-null Assertion (!!)

The non-null assertion operator (!!) tells the compiler to treat a nullable value as non-null. If the value is actually null at runtime, it throws a NullPointerException immediately. It should be used sparingly and only when the developer is certain the value cannot be null — for example, after an explicit null check in a different scope.

2.2 Kotlin Code Examples

Safe Call with Elvis Operator

```
val name: String? = null

// Safe call (?.) returns null if name is null
// Elvis operator (?:) substitutes 0 when the result is null
val length = name?.length ?: 0
println(length)  // Output: 0

// Chained safe calls
val city = user?.address?.city ?: "Unknown city"
```

Explicit Null Check (equivalent to safe call)

```
val name: String? = "Guru"

// Explicit null check — equivalent to name?.length ?: 0
val nameLength = if (name != null) name.length else 0
println(nameLength)  // Output: 4

// Non-null assertion — use only when certain name is not null
val forcedLength = name!!.length  // throws NPE if name is null
```

2.3 Scenario — User Profile Data from Server

Consider a user profile object fetched from a remote API. Fields like email are guaranteed by the server contract and are always present. Fields like nickname, photoUrl, and bio are optional — users may not have filled them in, and the server returns null for unset fields. Without null safety, accessing nickname.length on a null value would crash the app with a NullPointerException. Kotlin's type system forces the developer to acknowledge at compile time that these fields may be null and to handle them explicitly, making the app resilient to missing data.

```
data class UserProfile(
    val email: String,        // non-nullable — always present
    val nickname: String?,    // nullable — may be null if not set
    val photoUrl: String?,    // nullable — may be null if no photo
    val bio: String?          // nullable — may be null if not set
)

fun displayProfile(user: UserProfile) {
    val displayName = user.nickname ?: "Guest User"
    val profileImage = user.photoUrl ?: "default_avatar.png"
    println("Welcome, $displayName")
    println("Email: ${user.email}")  // safe — always non-null
}
```

2.4 Null-Safe Data Handling Table

Field Name	Type	Nullable	Safe Handling Strategy	UI Fallback
nickname	String?	Yes	Elvis operator: user.nickname ?: "Guest User"	Displays "Guest User" when null
photoUrl	String?	Yes	Safe call: user.photoUrl?.let { loadImage(it) }	Shows placeholder avatar image
bio	String?	Yes	Elvis operator: user.bio ?: "No bio provided"	Shows "No bio provided" when null
email	String	No	Always available — no null handling needed	Displays directly — guaranteed non-null

2.5 Jetpack Compose UI Example

```
@Composable
fun UserProfileScreen(user: UserProfile) {
    Column(modifier = Modifier.padding(16.dp)) {

        // Elvis operator provides fallback when nickname is null
        Text(
            text = user.nickname ?: "Guest User",
            style = MaterialTheme.typography.headlineMedium
        )

        // Safe call — only loads image if photoUrl is not null
        user.photoUrl?.let { url ->
            AsyncImage(model = url, contentDescription = "Profile photo")
        } ?: run {
            Image(painter = painterResource(R.drawable.avatar_placeholder),
                  contentDescription = "Default avatar")
        }

        // Non-nullable — always safe to display directly
        Text(text = "Email: ${user.email}")

        // Bio with fallback
        Text(text = user.bio ?: "No bio provided")
    }
}
```

[Insert UI sketch showing null-safe profile display with fallback values]

2.6 Reflection — Why Null Safety Prevents Crashes

Null safety is one of Kotlin's most practically valuable features because NullPointerExceptions are the single most common cause of Android app crashes. In languages without null safety, any object reference can be null at any time, and the only defence is developer discipline and runtime crashes. Kotlin's type system makes nullability a compile-time property: a String and a String? are genuinely different types, and the compiler refuses to compile code that dereferences a nullable type without explicit handling. This means that an entire class of potential crashes is caught before the app is even built, let alone shipped to users. The safe call operator and Elvis operator make null handling concise and expressive — user.nickname ?: "Guest User" is shorter, clearer, and safer than an equivalent multi-line if/else block in Java. The non-null assertion !! is intentionally verbose and alarming to discourage overuse: seeing !! in a code review is a signal to question whether the developer has truly verified that null cannot occur. Together, these features shift null handling from an afterthought to a first-class, compiler-enforced design decision.

Section 3: Feature Implementation Strategy

Feature Name: Event RSVP Form

Feature Description: Users enter the number of guests, select a meal preference from a dropdown, and use a toggle switch to accept or decline the invitation. The form applies null-safe validation to every field before allowing submission, and displays contextual error messages and confirmation text based on the user's choices.

3.1 Data & State Model

Variable	Type	Nullable	Description
guestsInput	String	No	Raw text from the guests TextField; converted to Int? before use
guestsCount	Int?	Yes	guestsInput.toIntOrNull() — null when the field is empty or non-numeric
mealPreference	String?	Yes	Selected meal from dropdown; null when the user has not chosen yet
rsvpStatus	Boolean	No	true = attending, false = declining. Non-nullable — always has a value

Compose State Variable Declarations

```
@Composable
fun RsvpForm() {
    // Raw text input — String, never null
    var guestsInput by remember { mutableStateOf("") }

    // Meal preference — nullable until user selects an option
    var mealPreference by remember { mutableStateOf<String?>(null) }

    // RSVP toggle — Boolean, starts as false (not attending)
    var rsvpStatus by remember { mutableStateOf(false) }

    // Derived nullable value — null if input is empty or non-numeric
    val guestsCount = guestsInput.toIntOrNull()
    // ...
}
```

3.2 Pseudocode Logic

```
// Derive nullable guests count from raw String input
val guestsCount = guestsInput.toIntOrNull()
```

```
// Meal preference with null-safe fallback
val selectedMeal = mealPreference ?: "Standard"

when {
    // User declined — no validation needed
    !rsvpStatus -> {
        showMessage("You have declined the invitation.")
    }

    // User accepted — validate guest count
    guestsCount == null -> {
        showError("Please enter a valid number of guests.")
    }

    guestsCount <= 0 -> {
        showError("Guest count must be greater than 0.")
    }

    // All valid — confirm RSVP
    else -> {
        showMessage(
            "RSVP confirmed! Guests: $guestsCount, Meal: $selectedMeal"
        )
    }
}
```

3.3 UI Layout Description

The RSVP form screen uses a vertically scrollable Column layout with the following components from top to bottom:

- Title Text — 'Event RSVP Form' heading at the top with padding
- TextField (guests) — numeric keyboard input bound to guestsInput state; shows inline error text below when guestsCount is null or zero
- Dropdown / ExposedDropdownMenuBox (meal preference) — bound to mealPreference state; defaults to 'Standard' via Elvis operator when no selection is made
- Row with Switch + Label — rsvpStatus toggle with 'Attending' label on the right; Switch is bound to rsvpStatus Boolean state
- Submit Button — enabled only when rsvpStatus is true and guestsCount is not null and guestsCount > 0; disabled styling applied via enabled = ... parameter
- Result Text — displays confirmation message or error message derived from the when expression above

[Insert UI wireframe sketch of the RSVP Form here]

3.4 Edge Case Handling Table

Edge Case	Expected Behaviour
Guests field is blank	guestsInput.toIntOrNull() returns null. Error message shown: 'Please enter a valid number of guests.' Submit button remains disabled.
Guests field contains 0	guestsCount == 0 triggers the guestsCount <= 0 branch. Error message: 'Guest count must be greater than 0.' Submission prevented.
Guests field contains negative number	guestsCount < 0 is caught by the guestsCount <= 0 branch. Same error message. Negative guest count is not a valid RSVP.
Meal preference not selected (null)	Elvis operator substitutes 'Standard': val selectedMeal = mealPreference ?: "Standard". Form can still submit; no error shown for this optional field.
User toggles rsvpStatus rapidly	Each toggle updates the Boolean state immediately. Compose recomposes the result Text and Submit button enabled state on each toggle change. No lag or stale state.
User declines (rsvpStatus = false)	The !rsvpStatus when branch fires. A decline message is shown immediately. Guest count and meal fields are visually dimmed (disabled or low opacity) since they are irrelevant.

3.5 Reflection — Combining Conditional Logic and Null Safety

Combining conditional logic and null safety in a UI form like the RSVP feature introduces several interrelated challenges. The first is the layered nature of validation: each field must be independently validated, but the final submission check must combine all validations simultaneously, which requires careful ordering of when branches to avoid masking one error with another. For example, checking rsvpStatus first before checking guestsCount ensures the user is not shown a guest count error when they have already declined — a detail that is easy to miss if conditions are written without a deliberate plan. The second challenge is the String-to-Int conversion: TextField always produces a String, but the guests count is an Int concept. The toIntOrNull() conversion introduces a nullable Int that must be handled at every point of use, threading nullability through the logic in a way that is simple in isolation but can become complex when nested inside conditional expressions. The third challenge is Compose's reactive model: because state updates trigger recomposition, a poorly ordered validation chain can cause error messages to flash briefly before the correct state is established, creating a confusing UI. Using a single when expression that maps the complete application state to a single result message — rather than multiple independent if/else checks each writing to different state variables — keeps the recomposition surface minimal and the UI consistently correct at all times.